

Low-Precision Inference & Deployment for EEG Deep Learning Models

Sapere Francesco Maria - ID: 25-945-999

19.08.2026



Outline

1. Project Overview & Goals
2. Models & Quantization
3. Deployment on Siracusa SoC
4. Hardware Results & Conclusions

Outline

1. Project Overview & Goals

2. Models & Quantization

3. Deployment on Siracusa SoC

4. Hardware Results & Conclusions

The Challenge...

- **Edge-BCI (Brain-Computer Interface):**

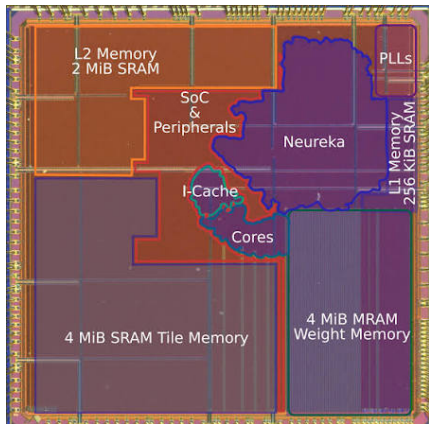
- **Requirements:**

- ▶ Real-time latency
 - ▶ Data privacy

⇒ Cloud offloading is unfeasible, let's do inference **on the edge!**

- **Ultra-low-power SoCs:**

- Very limited **on-chip SRAM capacity**
⇒ **Quantization** is needed;
 - Can exploit **dedicated NPUs**.



...& the Approach: Low-Precision & Co-Design

- **The Brevitas Framework (QAT):** Used **fake quantization** to simulate compression & numerical limits of the hardware during training (better accuracy);
- **Formats / Block Quantization:** FP32 → sub-byte formats (INT8, INT4, and **Microscaling MXFP4**).
- **Objective:** Evaluating the trade-off between architectural efficiency (e.g., MACs/cycle) and model classification quality.

```
class MXFloat4e2m1Weight(MXFloat8e4m3Weight):  
    bit_width = 4 # Shared exponent per block of 32 weights  
  
self.conv1 = qnn.QuantConv2d(...,  
    weight_quant=MXFloat4e2m1Weight,  
    input_quant=MXFloat8e4m3Act)
```

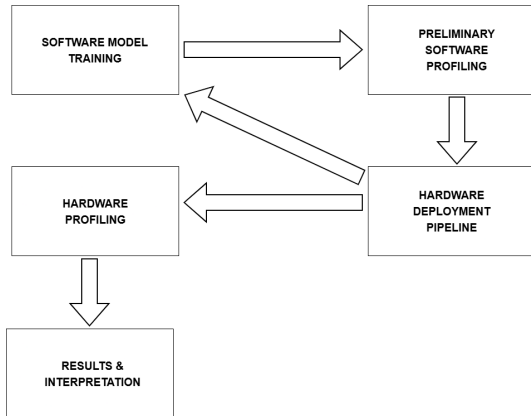
Workflow / Project Phases: Software

- **SW Model Training & Quantization**

- Topology Exploration (EEGNet vs. Conformer)
- Quantization-Aware Training (QAT)
- Microscaling (MX) & Mixed Precision

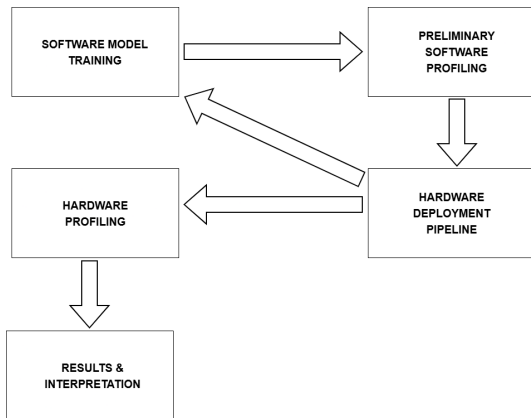
- **Preliminary (HW-Aware) Software Profiling**

- Theoretical Footprint Analysis
- Architecture Selection



Workflow / Project Phases: Hardware

- **Cycle-Accurate HW Profiling (GVSoC)**
 - L1/L2 Memory Utilization
 - Performance Metrics
- **Results & Architectural Interpretation**
 - Final Trade-offs
 - Lessons Learned



Outline

1. Project Overview & Goals
2. Models & Quantization
3. Deployment on Siracusa SoC
4. Hardware Results & Conclusions

Literature Review

- **Depthwise & separable convolutions** (after temporal-conv front-end)
⇒ number of parameters is drastically reduced [1];
- Use **self-attention** together with **convolutional front-end** ⇒ capture global correlations, overparametrized network [2].

Appendix A contains the **implementation choices** for the baseline models according to [1] & [2].

[1]: "EEGNet: A Compact Convolutional Neural Network for EEG-based Brain-Computer Interfaces", Lawhern et al.

[2]: "EEG Conformer: Convolutional Transformer for EEG Decoding and Visualization", Song et al.

Quantization Strategy

	EEGNet	EEG Conformer
Implementation	<code>nn.Conv2d</code> $\leftrightarrow$ <code>qnn.QuantConv2d</code> <code>nn.Linear</code> $\leftrightarrow$ <code>qnn.QuantLinear</code> (i.e., 1-to-1 PyTorch $\leftrightarrow$ Brevitas)	<code>nn.MultiheadAttention</code> $\implies$ <code>qnn.QuantMHA</code> (QAT of Q,K,V projections & intermediate MatMuls)
Precision & Formats	• Uniform INT8 (W8/A8) • INT32 accumulators (prevents MAC overflow)	• OCP-Standard MX Mixed-Precision • MXFloat4 (W4/block 32) & MXFloat8 (A8)
Scaling Factor: Type & Granularity	• Granularity: per-channel (W), per-tensor (A) • Type: Learned (via QAT)	• Granularity: per-block 32 (W), per-tensor (A) • Type: Dynamic (shared exp)
HW Alignment	• Native mapping to Siracusa SoC • Exploits N-EUREKA Accelerator & INT ALUs	• MX emulation on RISC-V INT ALUs • Complete FPU bypass (Integer-only inference)

Brevitas & Microscaling (MX) Implementation

1. Brevitas format extension to Sub-byte MX

Modified Brevitas injectors:

- Defined **custom MX4 format**;
- Re-used existing logic for **shared exponent calculation**

```
from brevitas.quant.experimental.mx_quant_ocp
    import MXFloat8e4m3Weight, MXFloat8e4m3Act

# Custom 4-bit MX Float Quantizer
class MXFloat4e2m1Weight(MXFloat8e4m3Weight):
    bit_width = 4 # Shared exp block size = 32
```

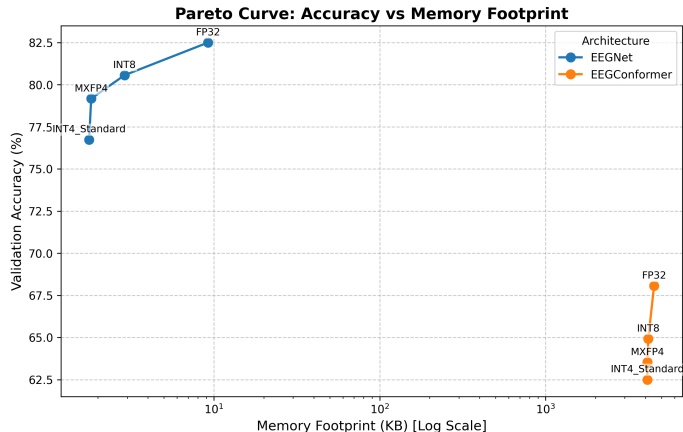
2. Replacing the layers

- `nn.Module` layers $\leftrightarrow$ `qnn.Module` layers
- Custom MX quantizers directly injected into the topology during training:

```
import brevitas.nn as qnn

# Quantization-Aware Spatial Convolution
self.spatial_conv = qnn.QuantConv2d(
    in_channels=1,
    out_channels=8,
    kernel_size=(1, 32),
    weight_quant=MXFloat4e2m1Weight, # MX4
    input_quant=MXFloat8e4m3Act      # MX8
)
```

Preliminary SW Profiling: Quantization Impact & Memory Footprint

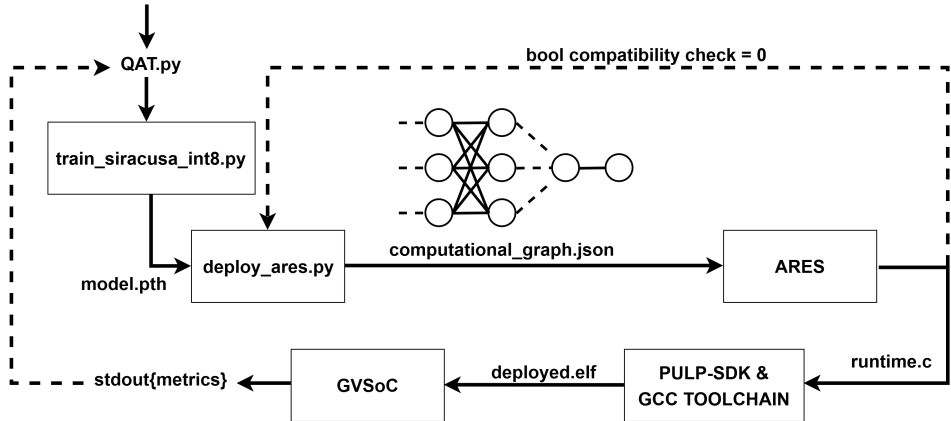


- **Memory Gain (FP32 $\rightarrow$ INT8)**
 - EEGNet: 9.2 KB $\rightarrow$ 2.3 KB
 - Conformer: 4600 KB $\rightarrow$ 1150 KB
- **Accuracy Dip:**
 - $\leq 6\%$ @ INT4;
 - $< 2\%$ @ INT8.
- **What to deploy then...?**
NEUREKA 
 - **Neureka** (Siracusa's Convolutional Accelerator).

Outline

1. Project Overview & Goals
2. Models & Quantization
3. Deployment on Siracusa SoC
4. Hardware Results & Conclusions

The Deployment Flow



Bypassing the FPU: Integer-Only Requantization

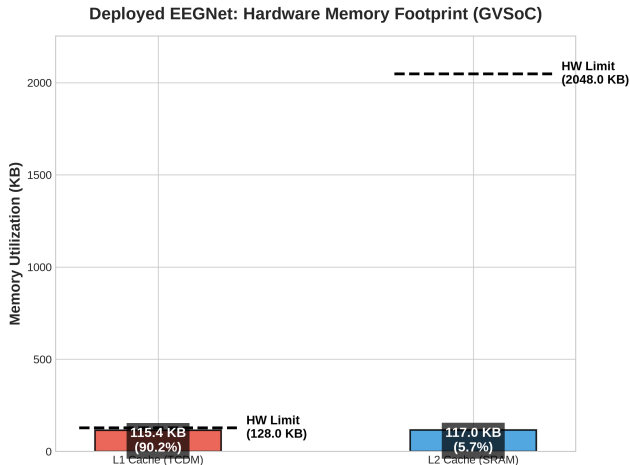
- **Scaling Factor**
 - QAT scaling requires FP math: $\text{Output} = \text{Acc}_{\text{INT32}} \times \text{Scale}_{\text{FP32}}$
- **Offline Approximation** (BrevitasExtractor)
 - $\text{Scale}_{\text{FP32}} \approx M \cdot 2^{-S}$
 - ▶ M : Integer Multiplier
 - ▶ S : Bit Shift
- **FPU** $\implies$ **1-cycle** integer **ALU** operation

$$\text{Output} = (\text{Acc}_{\text{INT32}} \times M) \gg S$$

Outline

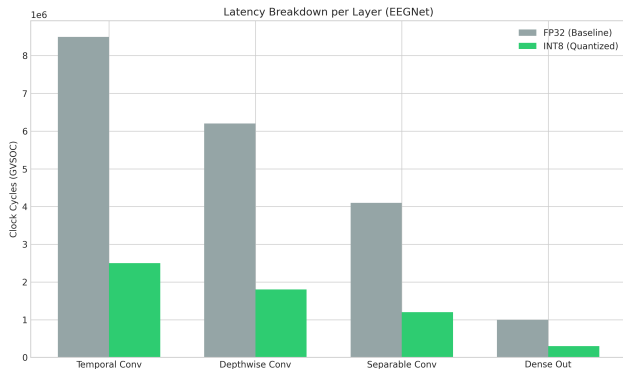
1. Project Overview & Goals
2. Models & Quantization
3. Deployment on Siracusa SoC
4. Hardware Results & Conclusions

Memory & Accuracy Metrics



- **L1 TCDM Utilization:** 115.4 KB
Safe Partitioning: aim to use < 50% of L1 TCDM.
- No need to use Siracusa's **4 MB WMEM**
- **Final Deployed Accuracy:** 80.56% (INT8)
- **Accuracy Dip:** < 2% vs. FP32 Baseline

Quantization Speedup Across Layers



- **Total Execution Time:**

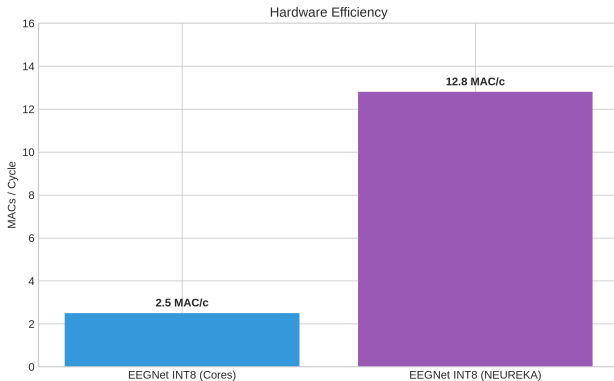
~19.84 Million Cycles

- **INT8 Impact:**

Up to $3\times$ **speedup** on early & compute-heavy layers (Temporal & Depthwise):

- Faster 8-bit data movement;
- INT8 vectorization
⇒ cores can process multiple MACs at once.

NEUREKA vs. RISC-V Cores



- **12.8 MACs/cycle**
(N-EUREKA Accelerator)
- **5.12 $\times$** speedup with respect to the baseline (RISC-V cores).

GVSoc Performance Overview

Metric	Deployed EEGNet (INT8)
Baseline Accuracy (FP32)	82.50%
Quantized Accuracy (INT8)	80.56%
Accuracy Dip	< 2%
Model Weights Footprint	9.2 KB $\rightarrow$ 2.3 KB
L1 Cache Utilization (TCDM)	115.4 KB < 128 KB
L2 Cache Utilization (SRAM)	117.0 KB < 2 MB
Total Execution Latency (Millions of Cycles)	$\sim$ 19.84
Execution Time ($f_{\text{clk}} = 100$ MHz)	198 ms
Peak Hardware Efficiency (N-EUREKA)	12.8 MACs/Cycle

Future Work

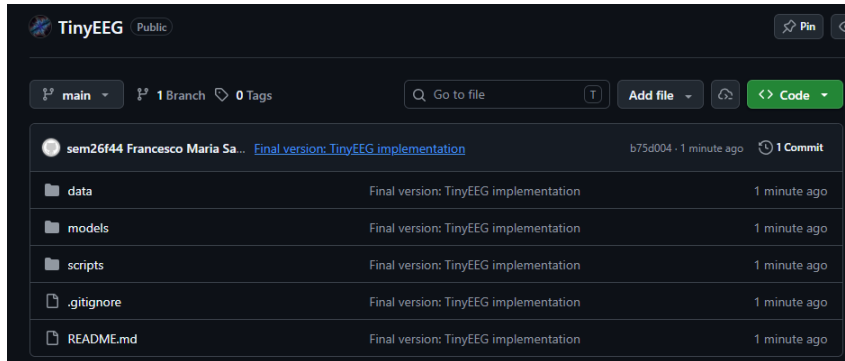
- Further **Design Space Exploration**

- EEGNet: INT8 $\rightarrow$ MX;
- **Custom Block Quantization:**
E vs. M trade-offs

- **Adaptive Quantization**

- **Architecture Specific Choices**

- Leverage **WMEM** (less banking conflicts, higher BW towards N-EUREKA)



Thank you for your attention!

Appendix A: Baseline Models Overview (1/2)

	EEGNet	EEG Conformer
Loss Function	Cross-Entropy	Cross-Entropy
Activation	ELU	ELU
Optimizer	Adam	Adam
Pooling Layers	Average Pooling	Average Pooling
Normalization	Batch Normalization	Batch Normalization
Dropout	0.5	0.1

Appendix A: Baseline Models Overview (2/2)

	EEGNet	EEG Conformer
Temporal Convolution	2D Conv (1×64)	2D Conv (1×25)
Spatial Convolution	Depthwise ($N_{\text{chans}} \times 1$) Separable (1×16)	2D Conv ($N_{\text{chans}} \times 1$)
Token Size	-	<code>d_model = 40</code>
Attention Heads	-	<code>nhead = 10</code>
Depth	-	<code>num_layers = 6</code>